



Основы радиоэлектроники и связи

Лекция 4. Файлы

Антон Штанюк

КТПП::ИРИТ::НГТУ / 2026-09-30

Файлы как модели источников

Файлы как модели источников

Информация

Для моделирования источников дискретных сообщений с заданными свойствами можно использовать файлы, содержащие данные. Содержимое файлов на низком уровне - байты, состоящие из 8 двоичных разрядов со значениями 0,1. Содержимое файлов на прикладном уровне предполагает кодирование и интерпретацию содержимого, исходя из заданного кода. Для кодирования текстовой информации широко используют следующие системы кодирования:

- ASCII
- UTF-8
- UTF-16 (Unicode)

ASCII

ASCII — это таблица кодировки символов, в которой каждой букве, числу или знаку соответствует определенное число. В стандартной таблице ASCII 128 символов, пронумерованных от 0 до 127. В них входят латинские буквы, цифры, знаки препинания и управляющие символы.

ASCII Table

Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char
0	0	0		32	20	40	(space)	64	40	100	@	96	60	140	`
1	1	1		33	21	41	!	65	41	101	A	97	61	141	a
2	2	2		34	22	42	"	66	42	102	B	98	62	142	b
3	3	3		35	23	43	#	67	43	103	C	99	63	143	c
4	4	4		36	24	44	\$	68	44	104	D	100	64	144	d
5	5	5		37	25	45	%	69	45	105	E	101	65	145	e
6	6	6		38	26	46	&	70	46	106	F	102	66	146	f
7	7	7		39	27	47	'	71	47	107	G	103	67	147	g
8	8	10		40	28	50	(	72	48	110	H	104	68	150	h
9	9	11		41	29	51	)	73	49	111	I	105	69	151	i
10	A	12		42	2A	52	*	74	4A	112	J	106	6A	152	j
11	B	13		43	2B	53	+	75	4B	113	K	107	6B	153	k
12	C	14		44	2C	54	,	76	4C	114	L	108	6C	154	l
13	D	15		45	2D	55	-	77	4D	115	M	109	6D	155	m
14	E	16		46	2E	56	.	78	4E	116	N	110	6E	156	n
15	F	17		47	2F	57	/	79	4F	117	O	111	6F	157	o
16	10	20		48	30	60	0	80	50	120	P	112	70	160	p
17	11	21		49	31	61	1	81	51	121	Q	113	71	161	q
18	12	22		50	32	62	2	82	52	122	R	114	72	162	r
19	13	23		51	33	63	3	83	53	123	S	115	73	163	s
20	14	24		52	34	64	4	84	54	124	T	116	74	164	t
21	15	25		53	35	65	5	85	55	125	U	117	75	165	u
22	16	26		54	36	66	6	86	56	126	V	118	76	166	v
23	17	27		55	37	67	7	87	57	127	W	119	77	167	w
24	18	30		56	38	70	8	88	58	130	X	120	78	170	x
25	19	31		57	39	71	9	89	59	131	Y	121	79	171	y
26	1A	32		58	3A	72	:	90	5A	132	Z	122	7A	172	z
27	1B	33		59	3B	73	;	91	5B	133	[	123	7B	173	{
28	1C	34		60	3C	74	<	92	5C	134	\	124	7C	174	
29	1D	35		61	3D	75	=	93	5D	135	]	125	7D	175	}
30	1E	36		62	3E	76	>	94	5E	136	^	126	7E	176	~
31	1F	37		63	3F	77	?	95	5F	137	_	127	7F	177	

ASCII

ASCII использует двоичный код одинаковой длины (7 бит), что позволяет добавить еще один вакантный бит и закодировать дополнительные 128 символов (расширенный ASCII). Исторически, в этом расширении хранились коды символы кириллицы (а также символы псевдографики, греческие, немецкие умляуты и т.п.)

ASCII

Преимуществом ASCII кода является выравнивание кодов на границы байт в памяти и соответствие: 1 символ - 1 байт памяти для хранения.

Пример генератора случайной строки

```
import string  
import random
```

```
random_string = ''.join(random.choices(string.ascii_letters, k=8))  
print(random_string)
```

Формирование файла из случайных латинских СИМВОЛОВ

```
# Количество символов для записи
num_chars = 100

# Выборка всех латинских букв (строчные и заглавные)
letters = string.ascii_letters # ascii_uppercase и ascii_lowercase

# Генерация случайных символов
random_data = "".join(random.choice(letters) for _ in range(num_chars))

# Запись в файл
with open("random_chars.txt", "w", encoding="ascii") as f:
    f.write(random_data)
```

Чтение файла и вывод двоичных кодов

```
def char_to_binary(char: str) -> str:
    return format(ord(char), '08b') # ровно 8 разрядов

with open('random_chars.txt', 'r', encoding='ascii') as file:
    while True:
        # Читаем ровно один символ
        char = file.read(1)
        # Если символ пустой,
        ^I^I# значит достигнут конец файла (EOF)
        if not char:
            break
        # Обрабатываем каждый символ
        print(char, end='-')
        print(char_to_binary(char))
```

UTF-8

Наиболее широко распространенный формат кодирования переменной длины, имеющий обратную совместимость со стандартным ASCII. Стандартный ASCII занимает в памяти 1 байт, расширенный - 2 байта (как правило).

Чтение файла в кодировке UTF-8

```
with open('example.txt', 'rb') as f:  
    content = f.read()  
    for byte in content:  
        print(f'{byte:08b}', end=' ')
```

Результат

```
11010000 10010000
11010000 10011111
11010000 10100000
11010000 10010100
...
```

Подсчет частоты встречаемости символов в файле

```
from collections import Counter

def count_characters(filename):
    # Открываем файл с кодировкой
    try:
        with open(filename, 'r', encoding='ascii') as file:
            text = file.read()
    except FileNotFoundError:
        print("Файл не найден.")
        return
    # Считаем частоту каждого символа
    char_counts = Counter(text)
    ...
```

Подсчет частоты встречаемости символов в файле

```
# Выводим результаты
print(f"Всего символов в файле: {len(text)}")
print("Частота встречаемости символов:")
for char, count in char_counts.most_common():
    # Заменяем перенос строки для красивого вывода
    display_char = repr(char) if char.strip() == ' ' else char
    print(f"{display_char}: {count}")

# Укажите имя вашего файла
count_characters('random_chars.txt')
```

Список источников I